

Introduction to Docker – Containerisation Made Simple

Welcome to your journey into containerisation technology that's revolutionising how we build, ship, and run applications.

by Naresh Chaurasia



Welcome & Agenda

01

What is Docker?

Understanding the fundamentals of containerisation

03

Key concepts: Images, Containers, Volumes, Networks

Core components that make Docker work

05

Use cases in real world

How Docker is transforming development

02

Why use Docker?

Exploring the benefits and advantages

04

Hands-on examples

Practical demonstrations and commands

06

Q&A

Your questions and discussion

The Problem Before Docker

Traditional deployment challenges



Works on my machine issue

Applications behaving differently across environments



Slow setup and scaling

Time-consuming deployment and configuration processes

Need for lightweight, portable environments



Heavy virtual machines (VMs)

Resource-intensive solutions with full operating systems



What is Docker?

Open-source platform for containerisation

Free technology that anyone can use and contribute to

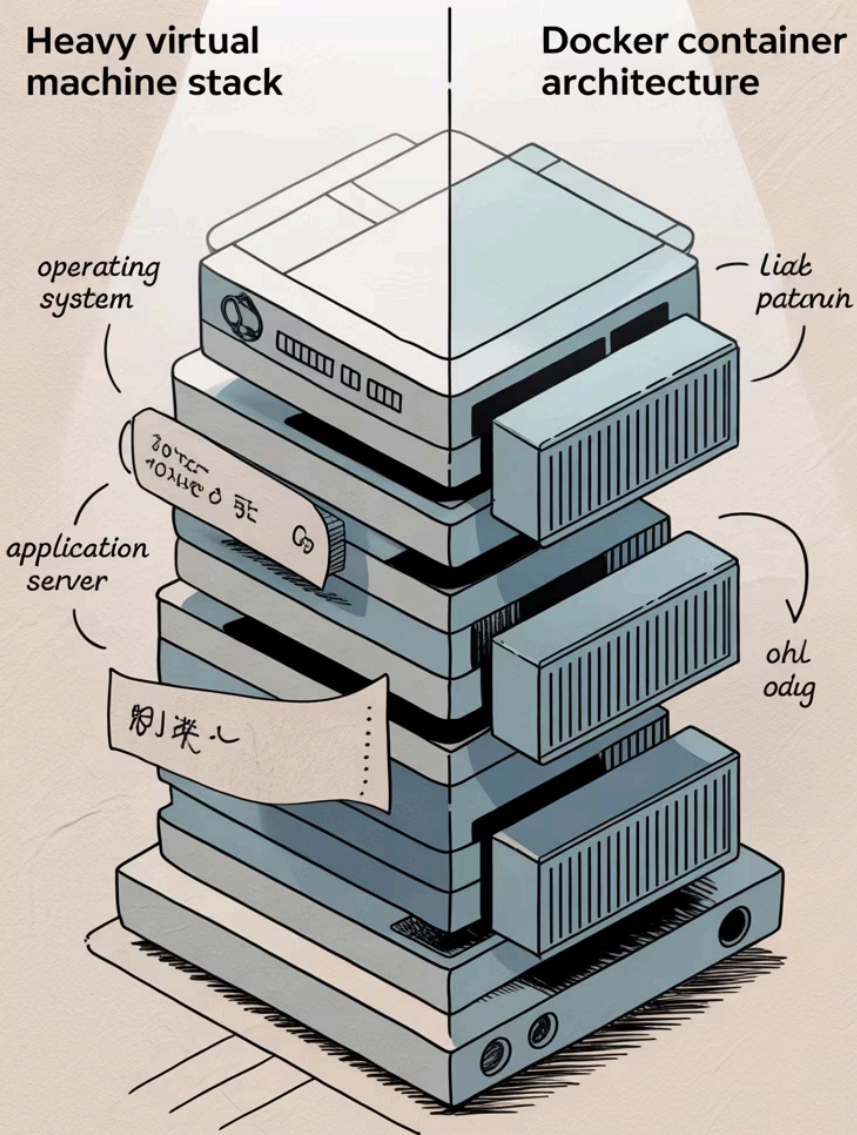
Packages applications with dependencies

Everything your app needs bundled together

Run anywhere: laptop, server, cloud

True portability across different environments





Docker vs Virtual Machines

Virtual Machines

- VM: Full OS, heavy, slow startup
- Resource intensive
- Longer boot times
- Larger storage requirements

Docker Containers

- Docker: Lightweight containers, share OS kernel, fast startup
- Efficient resource usage
- Instant startup
- Minimal storage footprint

Key Concepts in Docker



Image → Blueprint of an app
Read-only template containing
application code and dependencies



Container → Running instance of
an image
Executable package that runs your
application



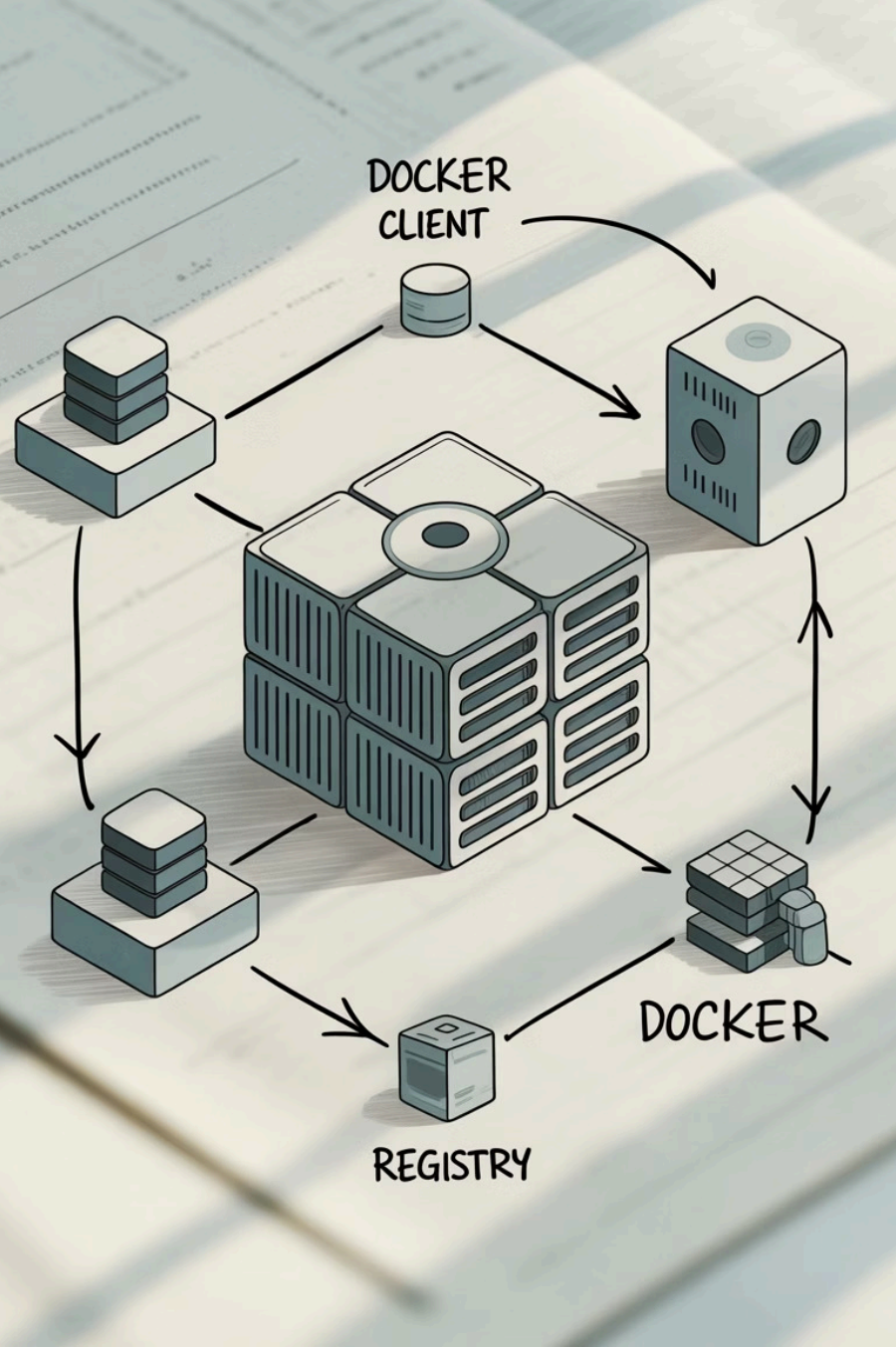
Dockerfile → Instructions to build
images
Text file with commands to
assemble an image



Registry → Store images (Docker
Hub)
Repository for sharing and
distributing images



Volumes & Networks → Data
persistence and communication
Managing data storage and
container connectivity



Docker Architecture

Docker Engine (Daemon + CLI + API)

Core runtime that manages containers, images, and networks

Client-server model

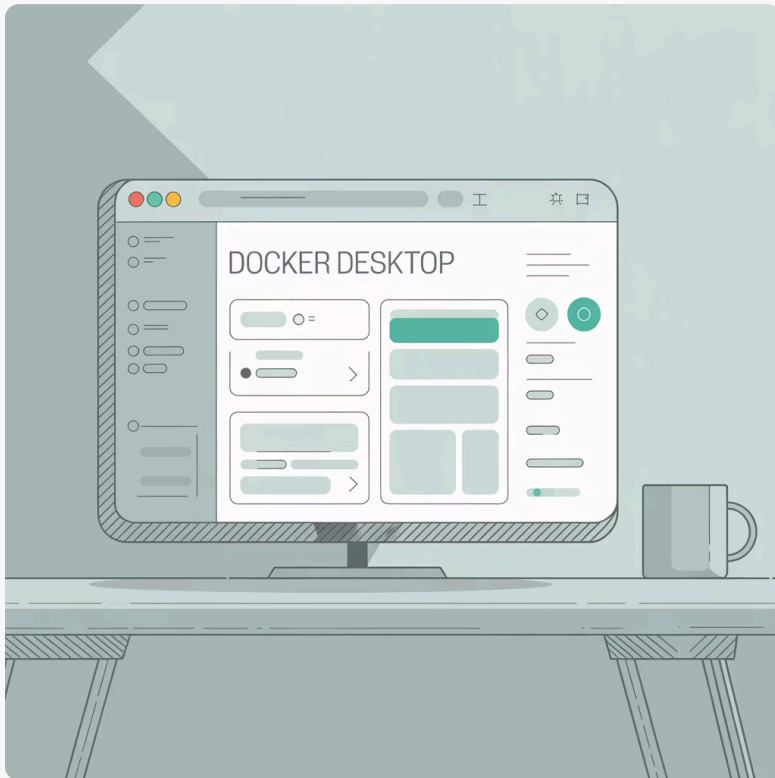
Docker client communicates with Docker daemon via REST API

Diagram showing how it works

Getting Started with Docker

Install Docker Desktop

Download and install from Docker's official website for your operating system.



Basic commands:

```
docker run hello-world
```

Test your Docker installation

```
docker ps
```

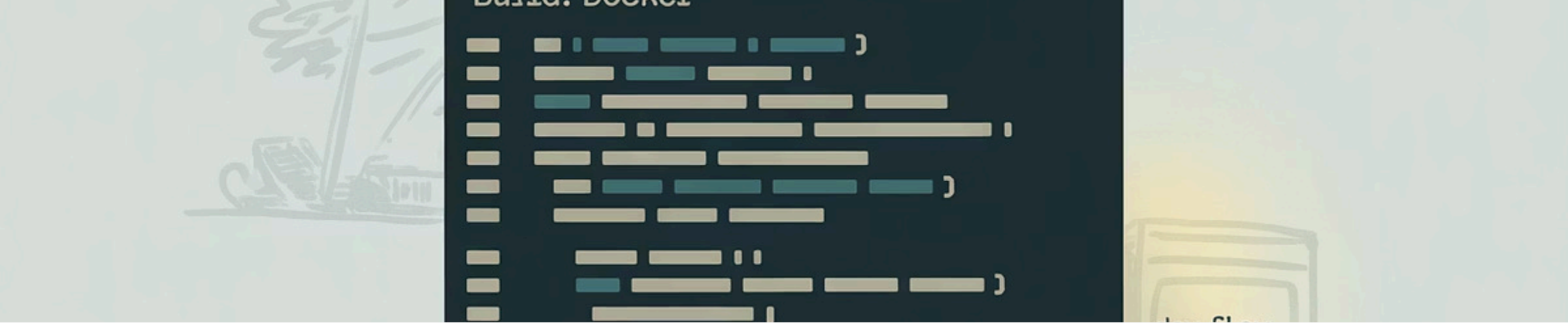
List running containers

```
docker stop <id>
```

Stop a running container

```
docker images
```

List available images



Building Your Own Image

Sample Dockerfile (e.g., Python app)

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build the image:

```
docker build -t myapp .
```

Run the container:

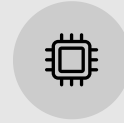
```
docker run -p 5000:5000 myapp
```

Docker Use Cases



Web application development

Consistent development environments across teams



Microservices

Deploy and scale individual service components



Continuous Integration/Continuous Deployment (CI/CD)

Automated testing and deployment pipelines



Cloud portability

Move applications seamlessly between cloud providers



Demo Plan (Optional)



Run hello-world

Verify Docker installation with the classic first container



Run nginx container and access via browser

Deploy a web server and see it running locally



Show Docker Hub pull

Demonstrate downloading images from the registry

Benefits of Docker & Next Steps

Benefits of Docker

- Portability
- Scalability
- Fast deployments
- Resource efficiency

Summary & Next Steps

- Containers simplify app deployment
- Learn more: Docker Compose, Kubernetes
- Resources: Docker docs, Play-with-Docker

Containers simplify app deployment

You're now ready to start your containerisation journey!